Lab-12: Running JUnit Tests with Maven/Gradle in Jenkins

Objective:
Configure Jenkins to build a Java project and execute JUnit test cases using both **Maven** and **Gradle**, and publish test results.

Lab Architecture
You will perform two parallel implementations:

1. Maven-based Java project
2. Gradle-based Java project

Both will:

- Execute JUnit tests
- Publish reports in Jenkins
- Fail the build on test failure

Prerequisites
Installed:

- Java 17+
- Maven
- Gradle
- Git
- Jenkins (running)

Verify:
java -version
mvn -version
gradle -version

PART A – Running JUnit Tests with Maven in Jenkins

🔹 **Step 1 – Create Maven Java Project**
mvn archetype:generate
Choose:

- GroupId: com.devops.lab
- ArtifactId: lab12-maven
- Packaging: jar
- Archetype: maven-archetype-quickstart

Move inside:
cd lab12-maven

🔹 **Step 2 – Update pom.xml for JUnit 5**

Replace dependencies with:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.1.2</version>
    </plugin>
  </plugins>
</build>
```

🔹 **Step 3 – Create Java Class**

src/main/java/com/devops/lab/Calculator.java

```
package com.devops.lab;

public class Calculator {
  public int multiply(int a, int b) {
    return a * b;
  }
}
```

🔹 **Step 4 – Create JUnit Test**

src/test/java/com/devops/lab/CalculatorTest.java

```
package com.devops.lab;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

  Calculator calculator = new Calculator();
```

```
    @Test
    void testMultiply() {
        assertEquals(20, calculator.multiply(4,5));
    }
}
```

🔹 Step 5 – Run Tests Locally

mvn clean test

Verify:

target/surefire-reports

Contains XML reports.

🔹 Step 6 – Push to Git

```
git init
git add .
git commit -m "Maven project with JUnit"
git remote add origin <repo-url>
git push -u origin main
```

🔹 Step 7 – Configure Jenkins (Maven)

7.1 Install Plugins

Ensure installed:

- Git Plugin
- Maven Integration Plugin
- JUnit Plugin
- Pipeline Plugin

7.2 Configure Tools

Manage Jenkins → Global Tool Configuration

Add:

- JDK 17
- Maven (Maven3)

🔹 Step 8 – Create Maven Pipeline Job

New Item → Pipeline → Name: Lab-12-Maven

Select: Pipeline script from SCM

Repository: <repo-url>

Branch: main

🔹 Step 9 – Create Jenkinsfile (Maven)

Add in project root:

```
pipeline {
  agent any

  tools {
    jdk 'JDK17'
    maven 'Maven3'
  }

  stages {

    stage('Checkout') {
      steps {
        git '<repo-url>'
      }
    }

    stage('Build & Test') {
      steps {
        sh 'mvn clean test'
      }
    }
  }

  post {
    always {
      junit '**/target/surefire-reports/*.xml'
    }
  }
}
```

Commit and push.

🔹 **Step 10 – Run Maven Pipeline**

Click Build Now.

Expected:

- Build Success
- Test report visible in Jenkins UI

PART B – Running JUnit Tests with Gradle in Jenkins

🔹 **Step 1 – Create Gradle Project**

gradle init
Choose:

- Application
- Java
- JUnit Jupiter
- Groovy DSL

Project name: lab12-gradle

🔹 **Step 2 – Verify build.gradle**

Ensure it contains:

```
plugins {
  id 'java'
}

repositories {
  mavenCentral()
}

dependencies {
  testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'
}

test {
  useJUnitPlatform()
}
```

🔹 **Step 3 – Create Java Class**

src/main/java/com/devops/lab/Calculator.java

```
package com.devops.lab;

public class Calculator {
  public int subtract(int a, int b) {
    return a - b;
  }
}
```

🔹 **Step 4 – Create JUnit Test**

src/test/java/com/devops/lab/CalculatorTest.java

```
package com.devops.lab;
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

  Calculator calculator = new Calculator();

  @Test
  void testSubtract() {
    assertEquals(5, calculator.subtract(10,5));
  }
}
```

🔹 **Step 5 – Run Tests Locally**

./gradlew test

Verify:

build/test-results/test

Contains XML reports.

🔹 **Step 6 – Push to Git**

```
git init
git add .
git commit -m "Gradle project with JUnit"
git remote add origin <repo-url>
git push -u origin main
```

🔹 **Step 7 – Configure Jenkins for Gradle**

Manage Jenkins → Global Tool Configuration

Add:

- JDK17
- Gradle (optional if not using wrapper)

If using Gradle Wrapper (gradlew), no global Gradle needed.

🔹 **Step 8 – Create Gradle Pipeline Job**

New Item → Pipeline

Name: Lab-12-Gradle

🔹 **Step 9 – Create Jenkinsfile (Gradle)**

```
pipeline {
  agent any
```

```
  tools {
    jdk 'JDK17'
  }

  stages {

    stage('Checkout') {
      steps {
        git '<repo-url>'
      }
    }

    stage('Build & Test') {
      steps {
        sh './gradlew clean test'
      }
    }
  }

  post {
    always {
      junit '**/build/test-results/test/*.xml'
    }
  }
}
```

Commit and push.

🔹 **Step 10 – Run Gradle Pipeline**

Click Build Now.

Verify:

- Test executed
- Report published in Jenkins

Validation: Force Failure

Modify test:

assertEquals(100, calculator.subtract(10,5));

Push and rerun.

Expected:

- Pipeline FAILED
- Red test in Jenkins

Demonstrates:

✓ CI Quality Gate

✓ Automated regression validation

Comparison: Maven vs Gradle in Jenkins

Aspect	Maven	Gradle
Default Test Plugin	Surefire	Built-in test task
Wrapper	Optional	Common
XML Report Path	target/surefire-reports	build/test-results
Speed	Slower (XML lifecycle)	Faster incremental

Lab Completion Checklist

- ✅ Maven JUnit tests executed in Jenkins
- ✅ Gradle JUnit tests executed in Jenkins
- ✅ Reports published
- ✅ Build fails on test failure
- ✅ Participants understand report path differences

Key Learning Outcomes

Participants understand:

- How Maven executes JUnit via Surefire
- How Gradle executes JUnit via test task
- How Jenkins parses XML reports
- How CI enforces quality gates
- Differences in report paths
- Importance of fail-fast pipelines